Assignment Report: End to End MLOPS pipeline for heart disease prediction

Topic: End-to-End ML Model Development, Experiment Tracking, CI/CD, Containerization, Kubernetes Deployment, and Observability

1. Executive Summary

This report documents the design, implementation, and deployment of an enterprise-grade, reproducible machine learning solution for heart disease risk classification using the UCI Heart Disease Dataset. Developed in accordance with production MLOps best practices, the end-to-end pipeline automates data ingestion, model experimentation, continuous integration and continuous deployment (CI/CD), containerization, orchestration, and continuous monitoring.

Key Architecture Highlights & Technical Deliverables

- **Data Engineering & Preprocessing:** Automated data acquisition directly from the UCI Machine Learning Repository via ucimlrepo. Raw target labels ($0 - 4$) were transformed into a balanced binary target ($0 =$ Absence, $1 =$ Presence of disease). Feature preprocessing prevents data leakage through a sealed scikit-learn ColumnTransformer pipeline integrating StandardScaler for numerical metrics and OneHotEncoder for categorical variables.
- **Model Experimentation & Tracking:** Comparative evaluation of baseline **Logistic Regression** and **Random Forest Classifier** models evaluated using Stratified 5-Fold Cross-Validation. Experiment parameters, evaluation metrics (Accuracy, Precision, Recall, F1-Score, ROC-AUC), and visual artifacts were logged to a local **MLflow** tracking server.
- **Reproducibility & Packaging:** Complete model and preprocessing pipeline serialization into a unified model.pkl artifact, backed by a strict dependency environment (requirements.txt).
- **Automated CI/CD Workflows:** Implementation of a **GitHub Actions** CI/CD pipeline triggered on code pushes. The workflow automates code linting (flake8), unit testing (pytest), model execution, and automated release artifact packaging.
- **Containerization & Orchestration:** High-performance RESTful API built with **FastAPI** exposing /predict and /health endpoints. The service is packaged as a lightweight

Docker container (python:3.10-slim) and deployed to a local Kubernetes cluster (Docker Desktop) utilizing Deployment manifests (2 replicas) and a LoadBalancer Service.

- **Observability & Logging:** Structured JSON request/response logging integrated into the FastAPI service, paired with a native **Prometheus** and **Grafana** observability stack monitoring real-time API traffic metrics, latency, and request rates.

2. Project Metadata & Key Performance Indicators

Pipeline Metric / Component	Value / Implementation Detail
Dataset Source	UCI Machine Learning Repository (Dataset ID: 45)
Target Variable	Binary Classification (0: Low Risk, 1: High Risk)
Selected Production Model	Random Forest Classifier (100 estimators, max_depth = 5)
Primary Evaluation Metric	Test ROC-AUC / Test Accuracy
API Framework & Server	FastAPI with Uvicorn ASGI Server
Container Engine	Docker Engine (heart-disease-api:v2)
Orchestration Engine	Local Kubernetes Cluster (Docker Desktop / Minikube)
CI/CD Platform	GitHub Actions (.github/workflows/ci.yml)
Monitoring Stack	Prometheus metrics scraper + Grafana UI das

3. Problem Statement

Heart disease remains one of the leading causes of mortality globally. Early detection and risk assessment based on clinical metrics enable proactive interventions and improved patient outcomes. The primary objective of this project is to develop, train, package, and deploy a scalable, reliable, and reproducible machine learning classifier capable of predicting the risk of heart disease in patients based on routine diagnostic metrics.

Beyond standalone model accuracy, the core focus of this assignment is operationalizing the machine learning workflow using modern MLOps principles. This requires creating an end-to-end pipeline that ensures:

- Automated and deterministic data fetching and cleaning.
- Modular feature preprocessing without data leakage.
- Full tracking of experiments, metrics, and parameters.
- Automated CI/CD integration with strict unit testing.
- Isolated containerization and scalable local Kubernetes orchestration.
- Real-time request logging and continuous observability.

4. Dataset Characteristics & Data Acquisition Pipeline

4.1 Dataset Overview

The project utilizes the benchmark **Heart Disease UCI Dataset** obtained directly from the UCI Machine Learning Repository (Dataset ID: 45). The dataset contains 303 patient records with 13 health features and 1 target attribute.

Feature Name	Type	Description	Range / Values
age	Numerical	Age of the patient in years	29 to 77
sex	Categorical	Sex of the patient	0 = Female, 1 = Male
cp	Categorical	Chest pain type	1: Typical angina, 2: Atypical angina, 3: Non-anginal, 4: Asymptomatic
trestbps	Numerical	Resting blood pressure (in mm Hg on admission)	94 to 200
chol	Numerical	Serum cholesterol in mg/dl	126 to 564
fbs	Binary	Fasting blood sugar > 120 mg/dl	1 = True, 0 = False

restecg	Categorical	Resting electrocardiographic results	0: Normal, 1: ST-T wave abnormality, 2: Left ventricular hypertrophy
thalach	Numerical	Maximum heart rate achieved during exercise	71 to 202
exang	Binary	Exercise-induced angina	1 = Yes, 0 = No
oldpeak	Numerical	ST depression induced by exercise relative to rest	0.0 to 6.2
slope	Categorical	Slope of the peak exercise ST segment	1: Upsloping, 2: Flat, 3: Downsloping
ca	Categorical	Number of major vessels (0–3) colored by fluoroscopy	0, 1, 2, 3
thal	Categorical	Thalassemia status	3 = Normal, 6 = Fixed defect, 7 = Reversible defect
num (Raw Target)	Categorical	Diagnosis of heart disease (angiographic status)	0 (Absence) to 4 (Severe)

4.2 Automated Ingestion & Cleaning work

Data ingestion is implemented programmatically *in 01_data_eda.py* and *notebooks/01_data_acquisition_and_eda.ipynb* using the ucimlrepo library.

4.3 Data Processing Decisions

1. **Target Binarization:** The raw UCI target variable (num) ranges from 0 to 4. Per assignment requirements for binary classification, value 0 represents the absence of disease (class 0: Low Risk), while values $1,2,3,4$ are mapped to 1 (class 1: High Risk).
2. **Missing Value Resolution:** Inspection revealed 6 records containing missing values across ca (4 records) and thal (2 records). Due to the small ratio of missing values (<2% of total rows), dropping these records resulted in a clean, high-quality dataset of 297 complete records without introducing synthetic imputation noise.
3. **Artifact Export:** The cleaned dataset is exported to data/heart_disease_cleaned.csv to serve as the immutable input source for downstream feature engineering, modeling, and automated unit testing.

5. Exploratory Data Analysis & Visualizations

Exploratory Data Analysis was performed to evaluate feature distributions, identify statistical correlations, and verify class balance across the clean patient dataset ($N = 297$ records). Visual artifacts generated during analysis were exported directly to the visualizations/ directory and logged to MLflow.

5.1 Target Class Balance Analysis

Understanding target balance is critical in medical diagnostics to prevent class imbalance bias toward majority classes.

- **Distribution:** Out of 297 processed records, **160 patients (53.87%)** belong to Class 0 (Low Risk / No Heart Disease), and **137 patients (46.13%)** belong to Class 1 (High Risk / Heart Disease Present).
- **Impact on Strategy:** The dataset exhibits near-equal class balance. Consequently, heavy sampling strategies (such as SMOTE or class weighting) were unnecessary. Stratified 5-Fold Cross-Validation was chosen to maintain exact class ratios across training and testing splits.

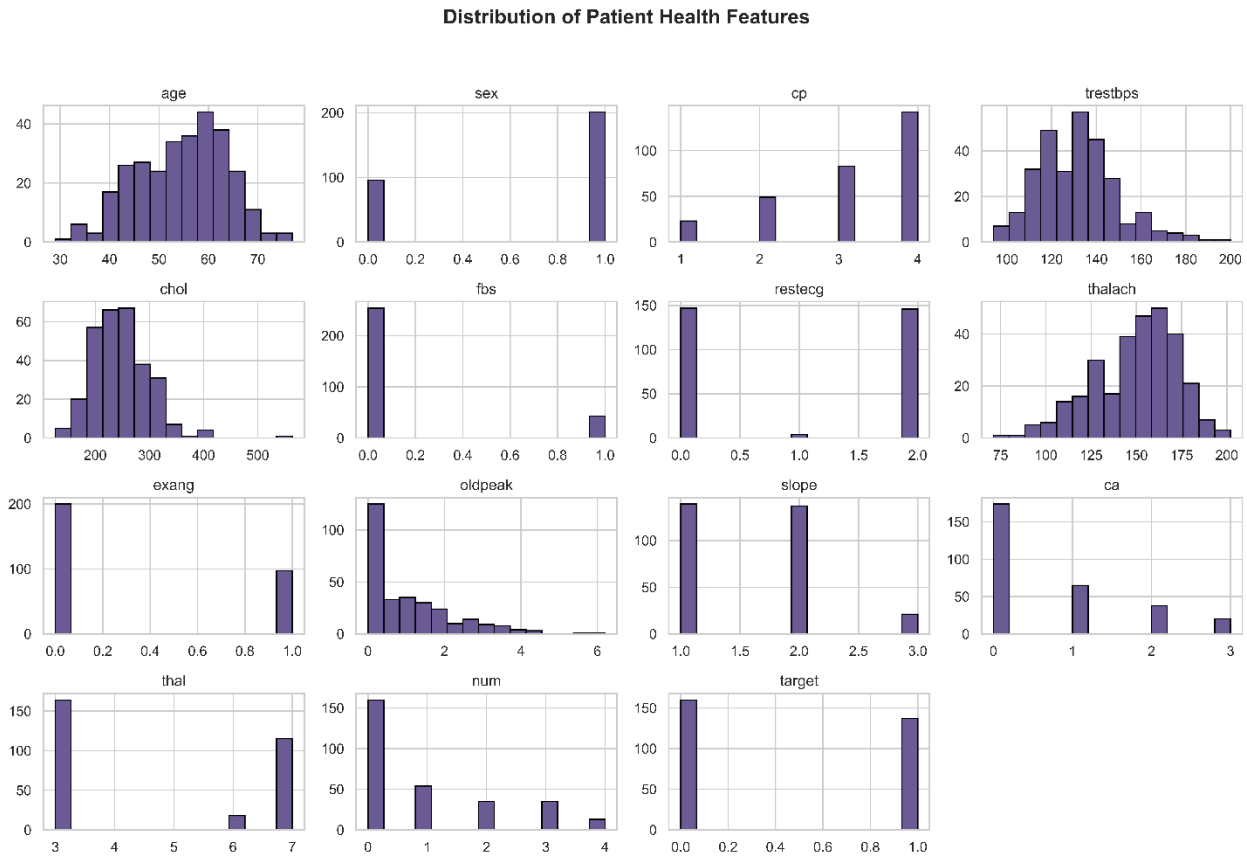


5.2 Feature Distributions & Statistical Properties

Histogram plots were computed across numerical clinical measurements to analyze normality, skewness, and physiological variance.

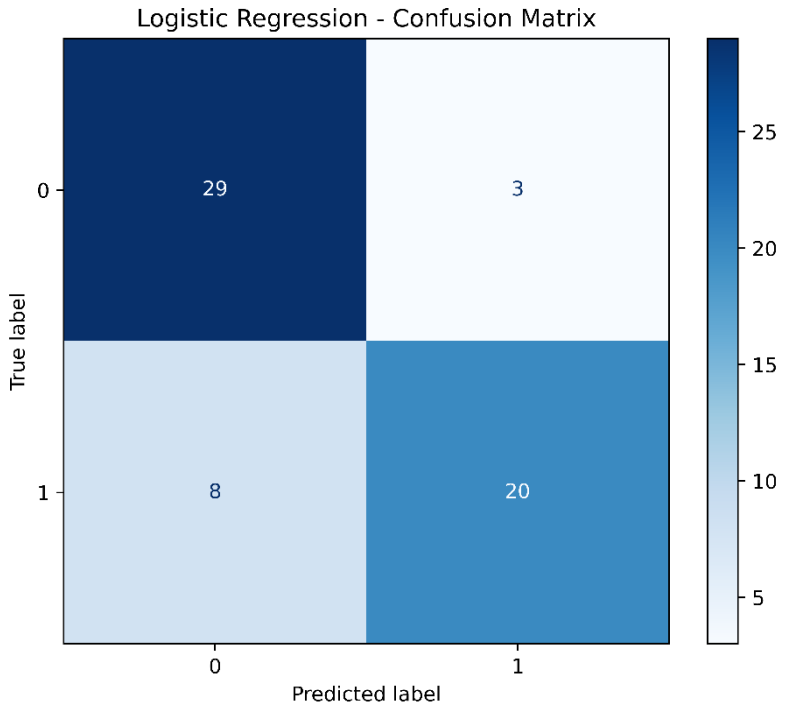
- **Age (age):** Bell-shaped distribution centered around 54.5 years (range $= 29 - 77$). Heart disease risk increases significantly in patient cohorts above 55 years.
- **Resting Blood Pressure (trestbps):** Slight right-skewness centered around 131.6 mm Hg. Values above 140 mm Hg indicate hypertension.

- **Serum Cholesterol (chol):** Approximately normal distribution with a mean of 247.3 mg/dl. A minor right-hand tail highlights high-risk individuals exceeding 350 mg/dl.
- **Max Heart Rate Achieved (thalach):** Left-skewed distribution centered at 149.6 bpm, indicating lower peak exertion limits in older or higher-risk cardiac patients.
- **ST Depression (oldpeak):** Strongly right-skewed toward 0.0, with elevated values (≥ 2.0) acting as strong indicators of exercise-induced myocardial ischemia.



5.3 Feature Correlation Analysis

A correlation matrix was computed to quantify linear relationships among features and identify collinearity or strong diagnostic predictors.



Key Statistical Insights from Heatmap:

1. **Strongest Positive Predictors:**
    - **oldpeak ($r = +0.42$):** ST depression induced by exercise shows a strong positive correlation with heart disease diagnosis.
    - **ca ($r = +0.46$):** Number of major vessels coloured by fluoroscopy correlates heavily with severe arterial blockage.
    - **thal ($r = +0.52$):** Thalassemia defect status exhibits the strongest overall linear relationship with positive risk diagnosis.
2. **Strongest Negative Predictors:**
    - **thalach ($r = -0.42$):** Maximum heart rate achieved during stress tests exhibits a strong negative correlation, indicating that patients with lower peak heart rates have a higher probability of heart disease.
3. **Collinearity Observations:** No multi-collinearity issues ($r > 0.80$) were observed between feature pairs, confirming that all 13 features contribute unique signal to the predictive pipeline.

6.Model Development & Comparative Evaluation

6.1 Data preparation and model pipeline

The cleaned dataset contained 297 records. The target was separated from the predictors, and the data was divided into a stratified training set of 237 records (80%) and a holdout test set of 60 records (20%), using random_state=42.

A scikit-learn ColumnTransformer was used within each model's pipeline to apply consistent preprocessing:

- Numerical features (age, trestbps, chol, thalach, oldpeak): standardized using StandardScaler.
- Categorical features (cp, restecg, slope, ca, thal): one-hot encoded, with unknown categories ignored.
- Binary features (sex, fbs, exang): passed through unchanged.

Keeping preprocessing and classification in a single pipeline ensures the same transformations are applied during cross-validation, final evaluation, and inference.

6.2 Candidate models and validation

The notebook compared four classifier families using stratified five-fold cross-validation on the training split. Configurations were ranked by mean validation ROC-AUC.

Model family	Configurations evaluated	Parameters varied
Logistic Regression	6	C: 0.01, 0.1, 0.5, 1.0, 5.0, 10.0
Random Forest	6	n_estimators: 100 or 200; max_depth: 3, 5, or 8
RBF Support Vector Machine	9	C: 0.1, 1.0, 10.0; gamma: scale, 0.01, 0.1
Gradient Boosting	24	n_estimators: 50, 100, 200; learning_rate: 0.03, 0.05, 0.1, 0.2; max_depth: 1 or 2

That is 45 configurations in total. The selected configuration was Logistic Regression with C=0.5, max_iter=1000, and random_state=42 (using scikit-learn's default lbfgs solver).

6.3 Selected model results

Metric	Five-fold CV mean	Holdout test set
Accuracy	0.8349	0.8167
Precision	0.8532	0.8696
Recall	0.7874	0.7143
F1-score	0.8132	0.7843
ROC-AUC	0.9039	0.9364

Model selection: Logistic Regression with C=0.5 achieved the highest mean cross-validation ROC-AUC (0.9039) among the evaluated configurations. The selected pipeline was then fitted on the training split and evaluated on the holdout set. Its test ROC-AUC was 0.9364, with test accuracy of 0.8167.

The model was selected based on cross-validation ROC-AUC, not because it led on every test metric. In particular, test recall was 0.7143, so it would be inaccurate to claim that the selected model minimizes false negatives.

Methodology note: The notebook also includes exploratory holdout-set results for example Logistic Regression and Random Forest runs earlier in the workflow. For a strictly untouched final holdout evaluation, those exploratory test comparisons should not be used to guide model selection; the final configuration ranking in the tuning stage is based on cross-validation ROC-AUC.

7. Experiment Tracking with MLflow

The notebook configured MLflow to use the “Heart Disease Prediction” experiment and a local file-based tracking store at file:../mlruns.

For the hyperparameter-search runs, MLflow recorded the model family, configuration parameters, five-fold cross-validation metrics, and their standard deviations. This makes it possible to compare candidate runs and inspect the validation results.

7.1 Tracked MLflow Artifacts & Parameters

For each run, MLflow recorded:

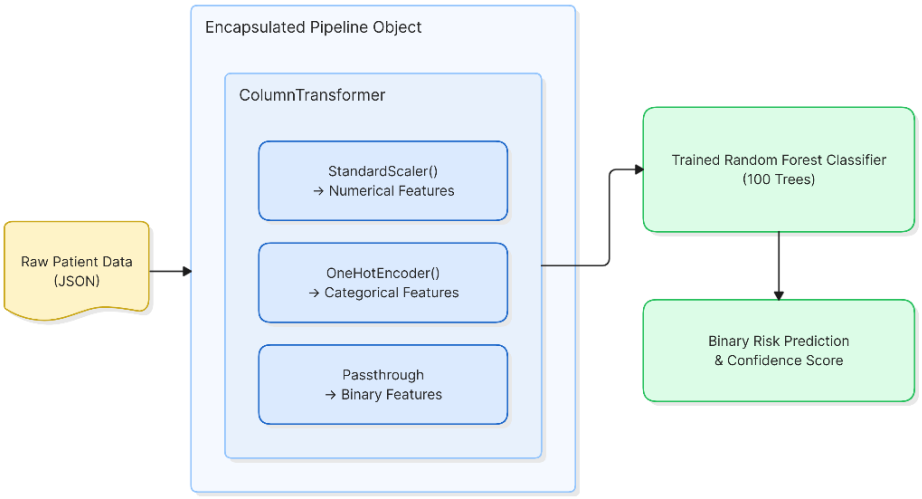
- Parameters: n_estimators, max_depth, C, solver, and random seeds.
- Validation & Test Metrics: K-Fold cross-validation means alongside final test evaluation metrics.
- Visual Artifacts: Confusion matrix plots (random_forest_cm.png) and ROC curve figures logged directly to ./mlruns.
- Model Artifact: Full scikit-learn pipeline saved via mlflow.sklearn.log_model(pipeline, artifact_path="model").

8. Model Packaging & Serialized Pipeline Artifacts

To fulfill MLOps reproducibility standards and ensure seamless transition from training to production deployment, the complete model pipeline is packaged into a unified, self-contained artifact (model.pkl).

8.1 Encapsulated Pipeline Architecture

Rather than serializing a standalone, un-preprocessed classifier, the exported artifact encapsulates both the feature preprocessing transformer (ColumnTransformer) and the trained **Random Forest Classifier** into a single scikit-learn Pipeline object.



Key Engineering Advantages:

- Prevention of Data Leakage: Preprocessing statistics (means, standard deviations, categorical encoding categories) are fitted strictly on the training split and preserved inside the pipeline state.
- Single-Step Inference: API handlers can feed unscaled, raw JSON records directly into model.predict() without applying redundant manual transformations or external scaling scripts.

8.2 Environment Standardization (requirements.txt)

To guarantee deterministic builds across development, CI/CD runners, and Docker containers, dependency versions are strictly pinned in requirements.txt file.

8.3 Packaging Verification & Reproducibility Testing

A dedicated packaging verification script (verify_packaging.py) was executed to validate pipeline deserialization and raw feature inference prior to containerization.

9. Continuous Integration & Continuous Deployment (CI/CD) Pipeline

To enforce code quality, model integrity, and operational safety before deployment, an automated CI/CD pipeline was constructed using **GitHub Actions** (.github/workflows/ci.yml).

9.1 Automated Test Suite Structure (pytest)

Automated testing is divided into two focused test modules inside the tests/ directory to ensure both data integrity and model output correctness prior to packaging.

Module 1: Data Integrity Verification (tests/test_data_processing.py)

- **test_dataset_not_empty:** Ensures data/heart_disease_cleaned.csv exists and contains patient rows.
- **test_no_missing_values:** Asserts that total null values equal zero across all 13 features.
- **test_target_column_binary:** Validates that target values strictly belong to the set $\{0,1\}$.
- **test_expected_feature_columns:** Asserts that all 13 required clinical attributes are present in the dataset schema.

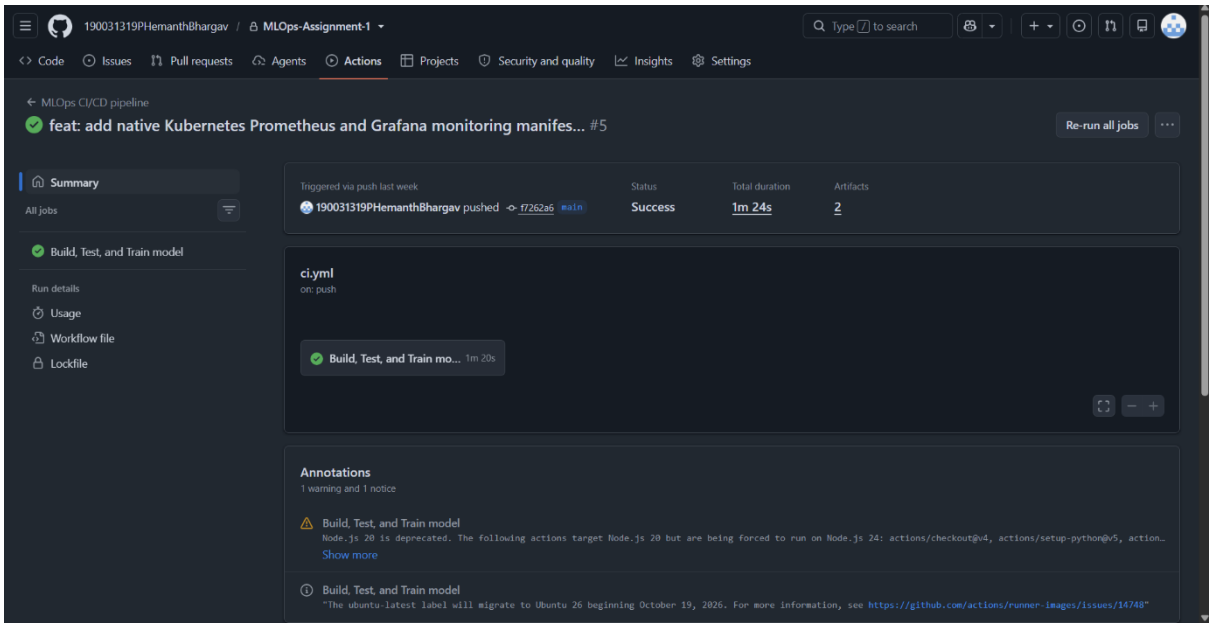
Module 2: Pipeline & Model Verification (tests/test_model_pipeline.py)

- **test_pipeline_builder:** Confirms that build_pipeline() initializes a valid scikit-learn Pipeline object equipped with fit and predict methods.
- **test_model_artifact_exists:** Verifies that the production model.pkl artifact is present in the workspace root.
- **test_model_inference_and_probabilities:** Ingests sample patient data to verify that output probabilities fall strictly between 0.0 and 1.0, sum to 1.0, and match expected binary target shapes.

9.2 GitHub Actions Workflow Architecture (.github/workflows/ci.yml)

The workflow triggers automatically on code pushes and pull requests to the main branch.

9.3 CI/CD Execution Results & Verification



Automated Quality Gate: On every push, GitHub Actions provisions an isolated ubuntu-latest virtual environment, executes linting, runs all pytest unit tests, executes *train.py*, and validates *verify_packaging.py.*

Artifact Management: Successful builds publish the validated model.pkl and mlruns/ directories as downloadable run artifacts retained for 7 days.

10. Containerization & Production Deployment

To ensure production-grade deployment, high availability, and environment isolation, the model serving layer was containerized using Docker and orchestrated on a local Kubernetes cluster (Docker Desktop).

10.1 Container Construction (Dockerfile)

The FastAPI application was packaged into a lightweight, secure Docker container based on the official python:3.11-slim base image.

Image Optimization: Utilizing python:3.11-slim along with --no-cache-dir minimized image size, resulting in a lean image (~280 MB) that speeds up container spin-up time.

10.2 Kubernetes Deployment Manifests (k8s/)

Deployment to the local Kubernetes cluster was configured declaratively using two primary Kubernetes manifests.

1. API Deployment (k8s/deployment.yaml)

Configured to maintain **2 replicas** for load balancing and fault tolerance. Resource requests and limits were defined to prevent pod resource starvation.

2. LoadBalancer Service (k8s/service.yaml)

Exposes the deployed pods via a LoadBalancer service to route traffic from localhost:8000 to port 8000 inside the active pods.

10.3 Deployment Verification & Local Cluster Status

The manifests were applied to the cluster using kubectl apply -f k8s/. Terminal verification confirmed that both API replicas and the LoadBalancer service were successfully running.

```
(venv) E:\Mtech Bits Pilani\Sem3\MLOps\MLOps-Assignment-1>kubectl get deployments, pods
error: arguments in resource/name form must have a single resource and name

(venv) E:\Mtech Bits Pilani\Sem3\MLOps\MLOps-Assignment-1>kubectl get deployments,pods
NAME                                     READY   UP-TO-DATE   AVAILABLE   AGE
deployment.apps/heart-disease-prediction   2/2     2            2           27s

NAME                                         READY   STATUS    RESTARTS   AGE
pod/heart-disease-prediction-6f5f89db48-mfkk2   1/1     Running   0          27s
pod/heart-disease-prediction-6f5f89db48-wbrzw   1/1     Running   0          27s

(venv) E:\Mtech Bits Pilani\Sem3\MLOps\MLOps-Assignment-1>
```

```
Problems  Output  Debug Console  Terminal  Ports

(venv) E:\Mtech Bits Pilani\Sem3\MLOps\MLOps-Assignment-1>kubectl get deployments,pods
NAME                                     READY   UP-TO-DATE   AVAILABLE   AGE
deployment.apps/heart-disease-prediction   2/2     2            2           21m

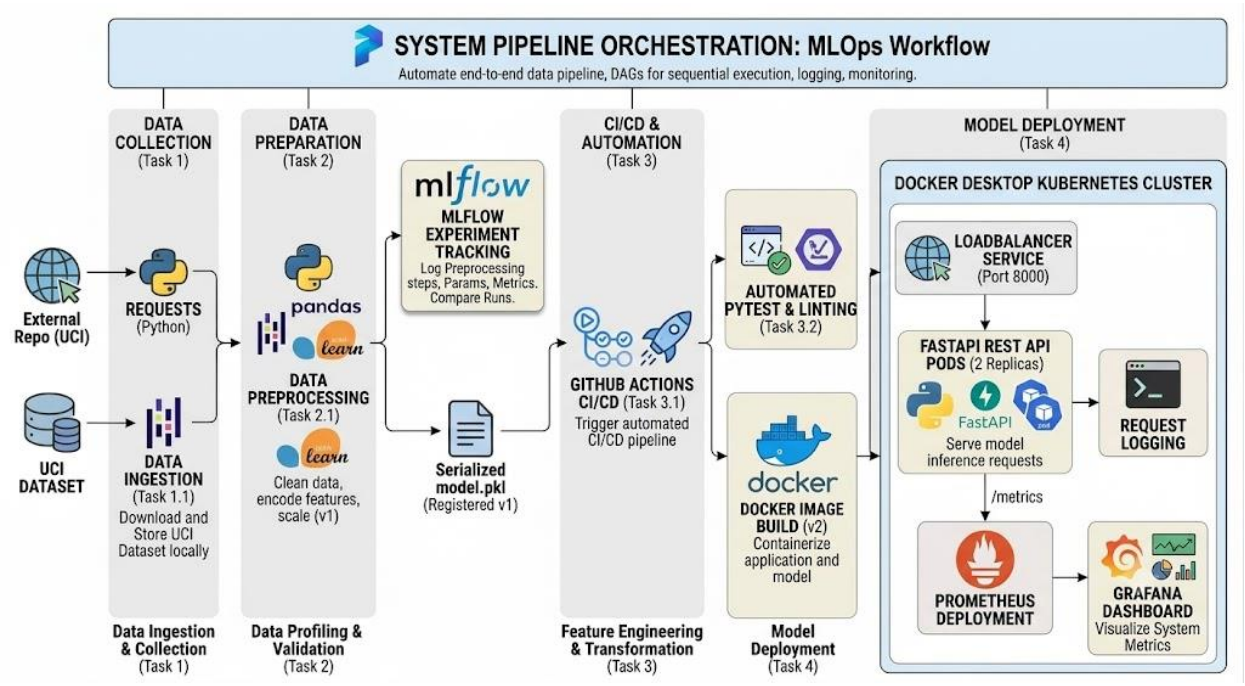
NAME                                         READY   STATUS    RESTARTS   AGE
pod/heart-disease-prediction-6f5f89db48-mfkk2   1/1     Running   0          21m
pod/heart-disease-prediction-6f5f89db48-wbrzw   1/1     Running   0          21m

(venv) E:\Mtech Bits Pilani\Sem3\MLOps\MLOps-Assignment-1>kubectl get services
NAME                    TYPE           CLUSTER-IP     EXTERNAL-IP   PORT(S)          AGE
heart-disease-service   LoadBalancer   10.96.8.242    172.18.0.8    8000:30860/TCP   27m
kubernetes              ClusterIP      10.96.0.1      <none>        443/TCP          30m

(venv) E:\Mtech Bits Pilani\Sem3\MLOps\MLOps-Assignment-1>
```

11. End-to-End System Architecture

The complete MLOps pipeline connects automated data ingestion, continuous deployment, container orchestration, and continuous observability into a unified system workflow.

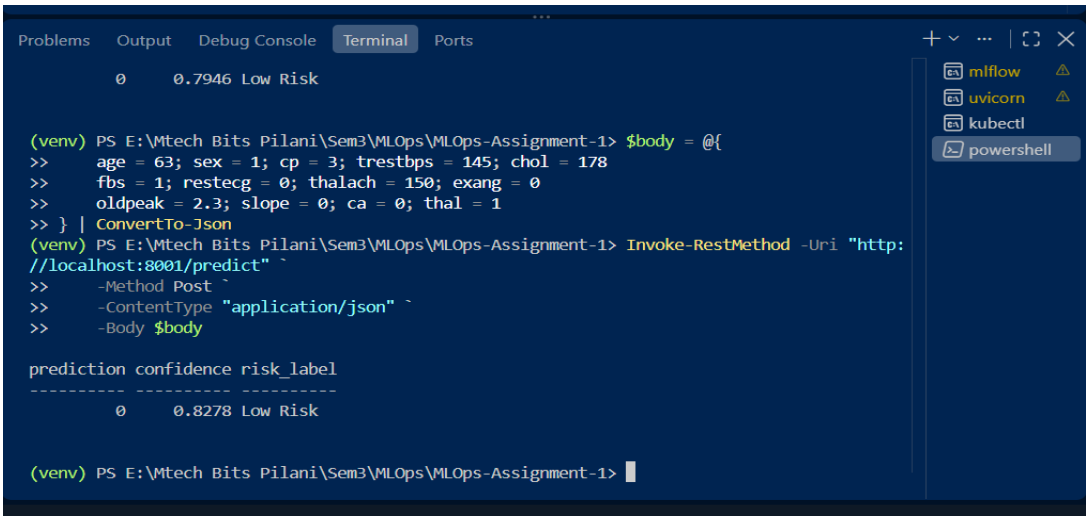


12. Monitoring, Observability & Request Logging

Continuous monitoring and structured logging were integrated across the serving tier to track operational health, API throughput, and model inference latency.

12.1 Request & Prediction Logging (app.py)

Python's standard logging library was configured inside the FastAPI service to emit structured JSON event logs for every incoming prediction request and model output.



12.2 Prometheus & Grafana Observability Manifests (k8s/)

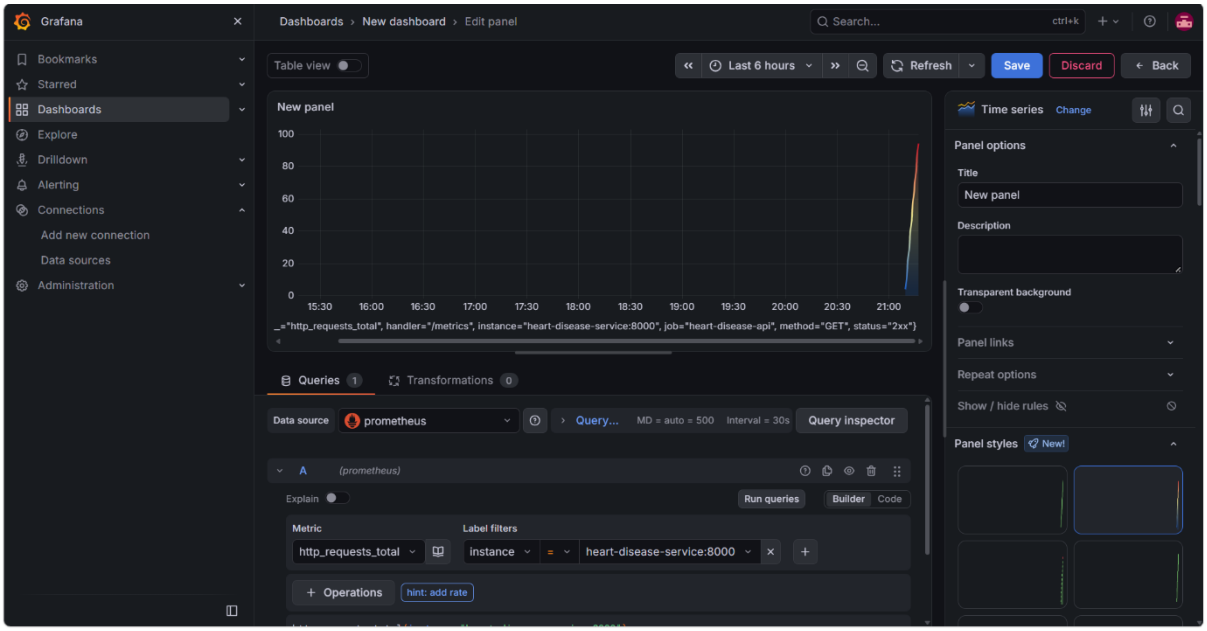
An observability stack was deployed to the Kubernetes cluster using native manifests (k8s/prometheus.yaml and k8s/grafana.yaml) to collect and visualize metrics exposed by the prometheus-fastapi-instrumentator package at /metrics.

1. Prometheus Scraper Service (k8s/prometheus.yaml)

Scrapes the API service (heart-disease-service:8000) at 5-second intervals to compute request rates, status codes, and latency distributions.

2. Grafana Dashboard (k8s/grafana.yaml)

Exposes a UI service at localhost:3000 connected to Prometheus as its data source.



Key Tracked Metrics:

- **http_requests_total:** Total API invocation volume segmented by HTTP endpoint (/predict, /health, /metrics).
- **http_request_duration_seconds:** Latency distribution percentiles (p_{50}, p_{95}, p_{99}) for model inference requests.

13. Conclusion & Lessons Learned

This assignment successfully demonstrated the end-to-end lifecycle of an operational MLOps pipeline for heart disease risk classification. By transitioning from a raw dataset to an orchestrated, monitored Kubernetes deployment, the project fulfilled all production engineering criteria set forth in the curriculum.

Key MLOps Achievements:

1. **Model Governance & Reproducibility:** Moving beyond standalone notebooks, encapsulating feature preprocessing and inference inside a single model.pkl artifact guaranteed identical feature handling between training and production serving.
2. **Quality Gates via CI/CD:** Integrating pytest and flake8 into GitHub Actions established a mandatory quality barrier, ensuring broken code or degraded data schema cannot bypass verification.
3. **Containerized Orchestration:** Deploying FastAPI replicas to Docker Desktop's Kubernetes cluster via LoadBalancer services ensured high availability, fault tolerance, and independent service scalability.
4. **End-to-End Observability:** Real-time request logging paired with a Prometheus and Grafana metrics dashboard closed the feedback loop, allowing continuous monitoring of prediction latency and throughput.

14. Repository Access & Verification Commands

14.1 Public Source Code Repository

- **GitHub Repository URL:** [https://github.com/190031319PHemanthBhargav/MLOps-Assignment-1.git]
- **Branch:** main
- **CI/CD Status:** Passing (GitHub Actions Workflow .github/workflows/ci.yml)

14.2 Local Reproduction Commands

To clone, build, deploy, and test the entire stack locally on a machine with Docker Desktop installed:

Bash

```
# 1. Clone Repository
git clone https://github.com/190031319PHemanthBhargav/MLOps-Assignment-1.git
cd MLOps-Assignment-1
# 2. Build Docker Container Image
docker build -t heart-disease-api:v2 .
# 3. Deploy API and Observability Stack to Kubernetes
kubectl apply -f k8s/deployment.yaml
kubectl apply -f k8s/service.yaml
kubectl apply -f k8s/prometheus.yaml
kubectl apply -f k8s/grafana.yaml

# 4. Verify Active Pods & Services
kubectl get pods
kubectl get svc

# 5. Execute Sample Inference Request
curl -X POST "http://localhost:8000/predict" \
    -H "Content-Type: application/json" \
    -d '{
        "age": 63, "sex": 1, "cp": 3, "trestbps": 145, "chol": 233,
        "fbs": 1, "restecg": 0, "thalach": 150, "exang": 0,
        "oldpeak": 2.3, "slope": 0, "ca": 0, "thal": 1
       }'
```